Lab 08: Textures and Transparency

CP479 Computer Graphics

October 16, 2026

An image model will give you a brick texture that is perfect as a flat picture, and it will still go wrong the moment it is wrapped around a model. The wall turns twice as bright as it should. The bumps light up from the wrong side. The floor melts into grey mush towards the horizon. Glass and smoke swap places as the camera turns. Chapter 9 explains each of these. In this lab you cause each one on purpose, measure it, and fix it in a shader.

The C++ is given. You edit **six shader files**, and every task is a few lines long. Most of the work is in the numbers. Every part has a panel that computes the right answer on the CPU, independently of your shader, and holds your output to it. Every part also has questions that only your own measurements can answer. That includes three places where your measurement and the chapter will disagree.

AI coding assistants are permitted and expected. You must be able to explain everything you ship. You must also keep an AI prompt log: for every prompt, record the idea behind it and how you verified the answer (see Section 8). An sRGB decode, a TBN matrix and a WBOIT shader are among the easiest code in graphics to find, so the marks are in the write-ups. In the last ten minutes of the session you will be asked to change one line in one of your shaders, live, and say what the picture will do before you press save.

1 Setup

Nothing new to install. The Vulkan SDK, CMake and GLFW you already have are the whole toolchain, and `glslc` comes with the SDK.

1.1 Get the skeleton and build it

`https://github.com/2110479-Computer-Graphics/Lab08-Texture`

Press *Use this template* to make your own copy, then clone that copy. Your work lives in your repository, so commit as you go. Then run *one* of these.

```
make info                                       # macOS, Linux
make.bat info                                   # Windows
cmake -S . -B build && cmake --build build --target info   # anywhere
```

The parts are `info srgb normalmap floor oit`. Each is a separate program, so if one fails to build the others still build. Every `.vert` and `.frag` in `shaders/` is compiled with `glslc`. The `.glsl` files are includes: they are never compiled on their own, but every shader that includes them depends on them, so saving `color.glsl` rebuilds every part.

1.2 What `make info` tells you

```
$ make info
  device          Apple M2 Pro
  driver          MoltenVK 1.4.1
  api             1.3.334
```

```
timestamps      1 ns per tick
anisotropy      samplerAnisotropy enabled, maxSamplerAnisotropy 16
blend           independentBlend yes (two attachments, two different blends)
srgb            R8G8B8A8_SRGB sampled with linear filtering: yes
float targets   blending into R16G16B16A16_SFLOAT yes, R16_SFLOAT yes, R8_UNORM yes
queryLod        textureQueryLod works: lod 2.00 (raw 2.00) for 4 texels a pixel,
    expected 2.00
glslc           /usr/local/bin/glslc
shaders         .../lab08-skeleton/shaders
color.glsl            TASK 1 incomplete
normalmap.vert        TASK 2 incomplete
...
```

The last block has one line per file you edit. `incomplete` means the file still contains a `TODO(TASK` marker. `done` means the marker is gone. This is a text search, not a grade: deleting the comment without writing the code also says `done`, and the panels will still say the picture is wrong. Two lines matter for this lab. *anisotropy* is the largest filter your GPU accepts, which is Part III. *timestamps* must be a number, or Parts III and IV cannot time anything. Keep this screenshot for your report.

1.3 Hot reload, keys and flags

Every part prints its panel, then opens a window. While the window is open, the host checks every file in `shaders/` four times a second. When you save a shader, it is recompiled and the picture updates without the window closing:

```
reloaded        color.glsl
compile error   color.glsl:18: error: 'pow' : no matching overloaded function found
```

If the compile fails, you get the message and the old shader keeps drawing. Fix the line, save again, and the picture updates.

Every part prints its own key legend when the window opens. The common ones, and the per-part ones you need for the write-ups:

Key or flag	What it does
`0 1 2 3`	the four views, the `mode` uniform
`S` / `Esc`	saves `report/<part>.png` / quits
left drag, scroll	orbit the camera, move it in and out (the panel ignores both); in Part I, left drag moves the split
`T`, `L`	Part I: the wall or the test tiles; pause the spotlights
`C K G`	Part II: bind the OpenGL or DirectX map; read it as OpenGL or DirectX (`u.knob`); bind it through the SRGB view
`L`, `[ ]`	Part II: pause the light; forced mip level of mode 3, ± 0.5
`F`, `T`	Part III: cycle the filter; 1 or 16 texture reads a pixel
`M W P R`	Part IV: method; the chapter's weight; 1000 or 10000 particles; reverse the draw order
`--headless`	no window, exit after the panel
`--save p.png`	write the frame the panel measured
`--mode m`, `--knob n`, `--knobf x`	set the uniforms directly
`--albedo f.png`	Part I: your own albedo texture instead of the test one
`--normal f.png`, `--dx`, `--srgb-normals`	Part II: your own normal map; start on the DirectX map; start with the SRGB view
`--filter name`	Part III: `nearest bilinear trilinear aniso4 aniso16`
`--method m`, `--particles n`	Part IV: `unsorted sorted wboit`; how many particles

```
make normalmap ARGS="--dx"
make floor ARGS="--filter aniso16 --mode 2"
make oit ARGS="--method sorted --particles 10000"
make.bat floor --filter nearest      # Windows takes them directly
```

The panel always measures at `time = 1.0` with the part's fixed camera, regardless of how you moved the window's camera. The reference images were rendered that way, which is why the panel can compare your picture against a file.

2 The skeleton

2.1 Files

- **The six files you edit**, and the only ones, all in `shaders/`:
  `color.glsl` (TASK 1), `normalmap.vert` and `normalmap.frag` (TASK 2),
  `miplevel.frag` (TASK 3), `wboit_accum.frag` and `wboit_composite.frag` (TASK 4).
- `shaders/params.glsl`: the uniform block, below, and the three texture slots. Every other shader is given: `srgb_wall.vert`, `srgb_wall.frag`, `srgb_tiles.frag`, `floor.vert`, `floor_helpers.glsl` (the colour ramps of Part III), `transparent.glsl` (the surface of every transparent fragment in Part IV), the `oit_*` shaders, and `fullscreen.vert`, a triangle that covers the target and needs no vertex buffer. Read the ones your part uses. They are short.
- `src/srgb.cpp` and its three siblings: one program per part. Each one makes its textures on the CPU, renders, probes pixels, computes the right answer independently on the CPU, times the passes, then opens the window. When a check value surprises you, read the code that printed it.
- `src/core/` and `src/utils/`: device, textures and samplers, render passes, pipelines, meshes, the frame loop with hot reload, the GPU timer, image comparison. Complete; do not edit.
- `reference/`: four PNGs from a correct solution. `report/`: where the `S` key saves PNGs. Created on first use, ignored by git.

2.2 The uniform block

There is one block of 256 bytes, the same for all four parts. It is declared once in `shaders/params.glsl` and mirrored byte for byte by `src/utils/Params.h`. Each part also binds up to three textures, `tex0` to `tex2`, at bindings 1 to 3 of the same set. A slot a part does not use holds a 1×1 white texture.

Part	`tex0`	`tex1`
I `srgb`	the albedo, `SRGB` view	the same bytes, `UNORM` view
II `normalmap`	the albedo, `SRGB`	the normal map, `UNORM`, mips
III `floor`	the floor, `SRGB`, mips	
IV `oit` composite	accumulation, `RGBA16F`	revealage, `R16F`

Member	Type	Offset	Size	What it is
`resolution`	`vec2`	0	8	the target, 800×600
`time`	`float`	8	4	seconds; the panel uses 1.0
`mode`	`uint`	12	4	0 final, 1 to 3 debug views
`light`	`vec4`	16	16	xyz direction *to* the light, world space; w intensity
`eye`	`vec4`	32	16	camera position, world space
`knob`	`uint`	48	4	Part II map convention, Part III stress taps, Part IV the chapter's weight
`knobf`	`float`	52	4	a forced mip level, Part II mode 3
`texSize`	`vec2`	56	8	size of `tex0`'s level 0, in texels
`model`	`mat4`	64	64	object to world
`view`	`mat4`	128	64	world to camera
`proj`	`mat4`	192	64	camera to clip
block size			**256**	

3 A texture is not a picture

Every part of this lab comes from one fact: a texture is an array of numbers, and the GPU does more to those numbers between the file and your shader than it looks like. Four ideas, one per part.

Two kinds of numbers (Part I). A byte in an albedo map is a *perceptual* value: sRGB spends more codes on dark tones, because that is where your eye tells them apart. Light does not add up in that space, so before any lighting the value has to be decoded to *linear* (§9.3, EQ-9.01). The exact curve is in two pieces, not $x^{2.2}$:

$$L(c) \;=\; \begin{cases} c/12.92 & c \le 0.04045 \\ \left(\dfrac{c+0.055}{1.055}\right)^{2.4} & c > 0.04045 \end{cases} \qquad c(L) \;=\; \begin{cases} 12.92\,L & L \le 0.0031308 \\ 1.055\,L^{1/2.4} - 0.055 & L > 0.0031308 \end{cases} \tag{1}$$

Byte 128 is $c = 0.502$, which decodes to $L = 0.2159$. Mid-grey reflects about a fifth of the light, not half. A normal map, a roughness map or a metallic map is *data*: a byte there means a number, so it must not be decoded. You choose which by the *format* you give the GPU. `R8G8B8A8_SRGB` makes the sampler apply $L(c)$ for you. `R8G8B8A8_UNORM` hands you the byte divided by 255. This lab's render target is `UNORM` too, so the final encode $c(L)$ is your job as well.

Tangent space (Part II). A normal map stores a direction relative to the surface: x along the texture's u axis, y along its v axis, z out of the surface. These are the tangent T, the bitangent B and the normal N. The shader decodes the byte to $[-1, 1]$ and rotates it into the world with the matrix whose columns are those three vectors:

$$\mathbf{n}_{\text{world}} \;=\; \text{normalize}\big(\,[\,T \;\; B \;\; N\,]\,(2\,\mathbf{t} - 1)\big) \tag{2}$$

The catch is which way B points. Some tools write green as "towards the top of the image" (the *OpenGL* convention). Others write it as "towards the bottom" (the *DirectX* convention, §9.3.1). Read a map in the wrong convention and every bump is lit from the wrong side, so it looks like a dent. This host builds B towards the top of the image, so an OpenGL map is read as it is and a DirectX map needs `n.y = -n.y`.

Which mip the GPU picks (Part III). When a texture is minified, one pixel covers many texels. A mip chain stores the texture pre-averaged at $\frac{1}{2}, \frac{1}{4}, \ldots$ of its size, $\frac{4}{3}$ of the original memory in total (§9.2.1). The GPU picks the level from how fast the texel coordinate $\mathbf{t} = \mathbf{uv} \cdot \texttt{texSize}$ changes from one pixel to the next:

$$\rho \;=\; \max\!\left(\left\|\frac{\partial \mathbf{t}}{\partial x}\right\|,\; \left\|\frac{\partial \mathbf{t}}{\partial y}\right\|\right), \qquad \lambda \;=\; \log_2 \rho \tag{3}$$

$\lambda = 0$ means one texel per pixel and $\lambda = 3$ means 8×8 texels per pixel. The derivatives are the same `dFdx`/`dFdy` you can call yourself. The GPU shades pixels in 2×2 quads and subtracts neighbours. *Trilinear* filtering blends the two levels on either side of λ. Look at a floor at a low angle, though, and one pixel's footprint on the texture is a long, thin ellipse. ρ takes the *long* axis, so the level is chosen for the long axis and the short axis is blurred along with it. *Anisotropic* filtering takes up to N samples along the long axis and chooses the level from the short one. The ratio of the two axes,

$$r \;=\; \frac{\max\left(\|\partial \mathbf{t}/\partial x\|,\, \|\partial \mathbf{t}/\partial y\|\right)}{\min\left(\|\partial \mathbf{t}/\partial x\|,\, \|\partial \mathbf{t}/\partial y\|\right)}, \tag{4}$$

shows where it matters: where r is 1, trilinear is already right, and where r is 8, trilinear blurs 8 times too much along one axis.

Over, and transparency without sorting (Part IV). One transparent layer of colour C and opacity α in front of whatever is already in the framebuffer gives the Porter-Duff *over* operator (EQ-9.03):

$$C_{\text{out}} \;=\; C\,\alpha \;+\; C_{\text{dst}}\,(1-\alpha) \tag{5}$$

Apply it once per layer, from the back to the front, and the result is exact. In any other order it is wrong. Sorting by object fails as soon as two objects cut through each other, and sorting every fragment is expensive. *Weighted blended OIT* (McGuire and Bavoil 2013, §9.5.3) gives up the exact answer for an order-free sum that the blend hardware can add up in any order:

$$C_{\text{out}} \;=\; \underbrace{\frac{\sum_i C_i\,\alpha_i\,w_i}{\sum_i \alpha_i\,w_i}}_{\text{weighted average colour}} \left(1 - \prod_i (1-\alpha_i)\right) \;+\; C_{\text{bg}} \prod_i (1-\alpha_i) \tag{6}$$

The product $\prod_i (1-\alpha_i)$ is the *revealage*: how much of the background still shows through. It does not depend on order, so that part is exact. Only the *split* of the covered part between the layers is approximate, and the weight w_i is a guess at how visible layer i is. Two layers of $\alpha = 0.5$, red in front and blue behind, over black: sorted *over* gives red 0.500, blue 0.250. With equal weights WBOIT gives 0.375 and 0.375. With the McGuire-Bavoil weight

$$w(z) \;=\; \text{clamp}\!\left(\frac{0.03}{10^{-5} + (z/200)^4},\; 10^{-2},\; 3 \times 10^3\right) \tag{7}$$

and the layers at $z = 2$ and 20 m, $w = 2997$ and 273, which gives 0.687 and 0.063. That is now wrong in the other direction. The weights that make it exact have ratio 2. Part IV's write-up asks why. z in Equation 7 is a distance in metres from the camera, and that detail is where the chapter's code goes wrong.

4 Part I: Two kinds of numbers

Run `make srgb`. A brick wall, lit by two spotlights, a warm one above and a cool one below. Their pools overlap in the middle. A vertical line splits the screen, and you can drag it with the

mouse. Left of the line is the correct pipeline. Right of it is a mistake, chosen with keys 0 to 3. Both sides read the same bytes, c, through the texture's `UNORM` view (`tex1`), and both call *your* functions:

Where	Displayed colour	What it is
left	$c(\,L(c_{\text{tex}}) \cdot (L_A + L_B)\,)$	correct: decode, light, encode
right, mode 0	$c_{\text{tex}} \cdot (c(L_A) + c(L_B))$	lighting in gamma space
right, mode 1	`albedo_check`$(L(c_{\text{tex}}))$	your QA view, unlit
right, mode 2	$c(\,c_{\text{tex}} \cdot (L_A + L_B)\,)$	no decode
right, mode 3	$L(c_{\text{tex}}) \cdot (L_A + L_B)$	no encode

L_A and L_B are the linear light of each spot at that point, and $L(\cdot)$ and $c(\cdot)$ are Equation 1. Mode 0 is how a whole pipeline goes wrong. Nothing is decoded or encoded, and each light's brightness was chosen *by eye*: whoever set it up turned each spot up until its centre looked right on screen. That is the $c(L_A)$. One spot on its own therefore looks the same on both sides. Watch what happens where the two pools meet, and at their edges. `srgb_wall.frag` is given. Read it: it is short.

Key `T` switches to the test pattern that the panel measures: three 256×256 tiles of a grey ramp and bricks. The host uploads the *same bytes twice*, as `tex0` in `R8G8B8A8_SRGB`, so the sampler decodes them, and as `tex1` in `R8G8B8A8_UNORM`, so it does not. Tile A shows `texture(tex0, uv)` (the hardware decodes), tile B `srgb_to_linear(texture(tex1, uv))` (you decode) and tile C `texture(tex1, uv)` (nobody decodes). In the tile view, mode 2 writes the linear values without encoding them. The panel probes that view.

4.1 TASK 1: `color.glsl`

Listing 1: `shaders/color.glsl`, the three TODOs

```
// TODO(TASK 1a)
// sRGB-encoded value (what the bytes of a colour texture mean) -> linear light.
// The exact piecewise curve, not pow(c, 2.2).
vec3 srgb_to_linear(vec3 c) {
  return c;
}

// TODO(TASK 1b)
// Linear light -> sRGB-encoded value (what the UNORM target and the screen
    expect).
// The exact inverse of srgb_to_linear.
vec3 linear_to_srgb(vec3 c) {
  return c;
}

// TODO(TASK 1c)
// A debug view for albedo textures: luminance of the LINEAR colour, too dark
// (< 0.04) shows blue, too bright (> 0.9) shows red, anything else shows that
// luminance as a grey, encoded for display.
vec3 albedo_check(vec3 lin) {
  return lin;
}
```

1a and 1b are the two halves of Equation 1, per channel. Write them without an `if`: `mix(a, b, step(edge, c))` or `lessThanEqual` picks between two vectors. 1c uses the luminance of the *linear* colour, $Y = 0.2126R + 0.7152G + 0.0722B$, and the chapter's PBR range (§9.6.1): $Y < 0.04$ is darker than charcoal and $Y > 0.9$ is brighter than fresh snow, and no real surface has either albedo. These limits are on *linear* values. Applied to raw bytes, they would flag a different set of pixels.

Check values. Work the first and last out before you run anything.

- **Texel 128 decodes to** 0.2159: $c = 128/255 = 0.50196$, then $((0.50196+0.055)/1.055)^{2.4} = 0.2159$. In mode 2 that is the byte $0.2159 \times 255 = 55$, on both A and B.
- **Ramp**: across all 256 grey levels, your decode and the hardware's differ by at most one byte. The GPU is allowed a little rounding of its own.
- **Round trip**: `linear_to_srgb(srgb_to_linear(x))` gives back every byte exactly, and so does `linear_to_srgb` of the hardware's decode.
- **No decode: texel 128 displays as 188.** Tile C treats 0.502 as linear, and your encode turns it into $1.055 \cdot 0.502^{1/2.4} - 0.055 = 0.737$. Its mid-grey is about three quarters of white.
- **Range**: the percentages of too-dark and too-bright pixels in the brick half match the CPU's.
- **Spotlights**: three pixels in the column $x = 390$, in spot A alone, in the overlap and in spot B alone, rendered once all correct (split at 800) and once all mode 0 (split at 0). Your left pipeline must match the CPU's to within 2 levels. The gamma pipeline matches yours in each spot alone, to within 4 levels: sRGB is not a pure power curve, so $c(ab)$ is only nearly $c(a)\,c(b)$. In the overlap the gamma pipeline is up to 69 levels brighter, and red clips at 255.

```
$ make srgb
  decode          texel 128 -> linear 0.2159 (byte 55)   yours 55   hardware 55   PASS
  ramp            max |yours - hardware| over 256 levels  1 (texel 97)   PASS (<= 1)
  roundtrip       max |encode(decode(x)) - x|  0 with your decode, 0 with the hardware's   PASS
  no decode       texel 128 displays as 188 (should be 128), cpu 188   PASS
  range           too dark 3.8% (cpu 3.8%)   too bright 2.6% (cpu 2.6%)   PASS
                  luminance of the linear colour, the albedo half of tile B
  per channel     the same limits on each channel alone (cpu): too dark 5.1%, too bright 2.6%
  spotlights      A warm (1.00, 0.62, 0.32) x 1.00 above, B cool (0.32, 0.58, 1.00) x 1.20 below
                  aimed at (0.50, 0.55) and (0.50, -0.55) m from 6 m out, no ambient
                  cone 4.5 to 10.2 deg, falloff 1 / d^2, both on the split column
                  yours  left of the split: linear_to_srgb(srgb_to_linear(raw) * (A + B))
                  gamma  right of it, mode 0: raw * (linear_to_srgb(A) + linear_to_srgb(B)),
                         each light tuned alone in gamma space
                  split at 800 for yours, at 0 for gamma, time 1; probes in column 390, on clay bricks
  spot A alone    (390, 223)  yours (172,  62,  34)  cpu (172,  62,  34)  PASS   gamma (172,  64,  38)
  overlap         (390, 296)  yours (186,  99,  82)  cpu (185, 100,  82)  PASS   gamma (255, 142, 110)
  spot B alone    (390, 374)  yours (102,  80,  76)  cpu (103,  81,  76)  PASS   gamma (104,  81,  75)
  gamma = yours?  A alone yes (max 4), overlap no: gamma up to +69 and clipped, B alone yes (max 2)
  image           MATCH (0.00% differ)   (wall, split 400, mode 0)
```

The `per channel` line is the chapter's version of the range check, computed on the CPU. Its too-dark number is 1.3 points higher than yours, and the difference is one brick. That brick is write-up 1.4.

4.2 Expected result

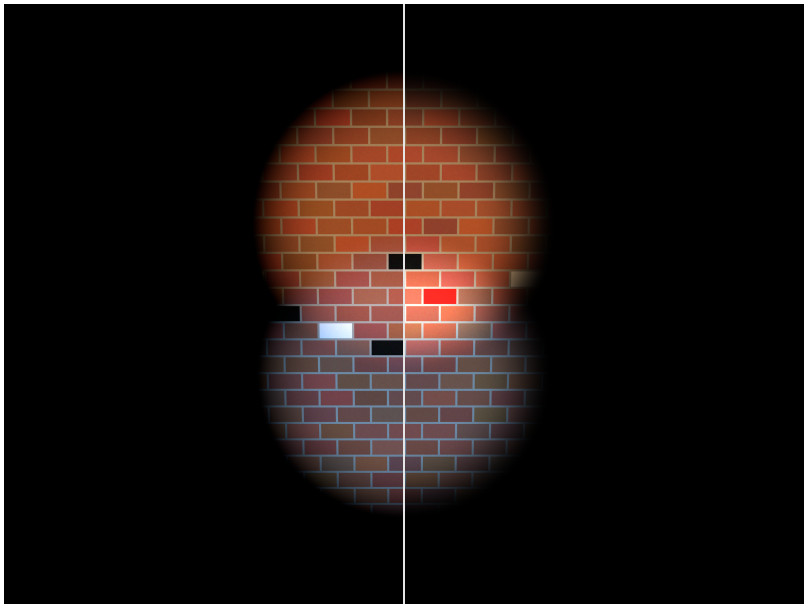


Figure 1: **Done, mode 0, split in the middle.** Each pool's centre is the same on both sides of the line. That is how the right side's lights were tuned. On the right the lens where the pools meet is hotter and its reds clip, and both pools fall off sooner towards their edges. Write-up 1.2 asks you to predict both effects in numbers.

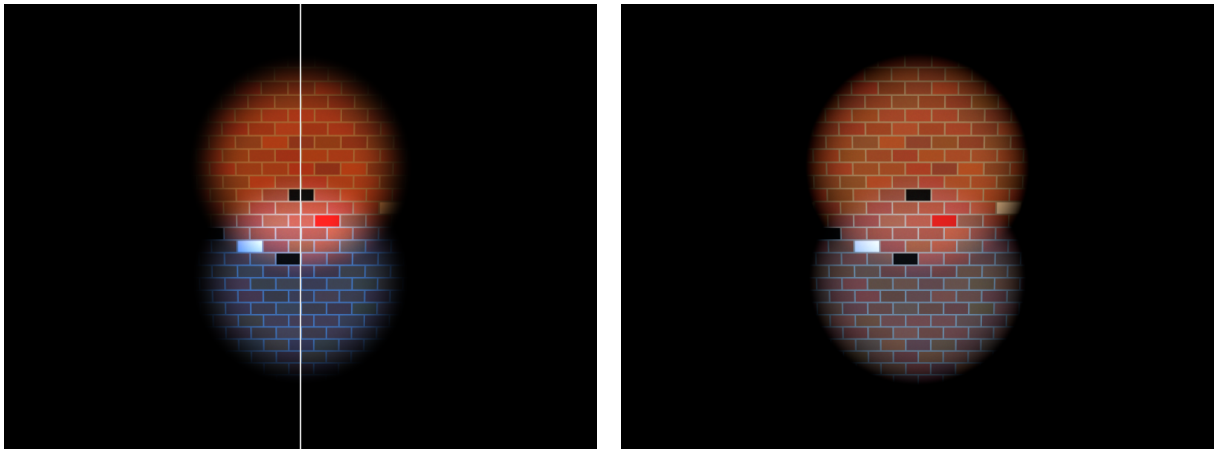


Figure 2: **Skeleton:** both of your functions return their input, so the left side is lit in gamma space too and the two halves are identical: saturated colours, pools that end abruptly. **Done, split dragged off to the right:** the correct pipeline everywhere, two soft pools and a pale lens where they meet.

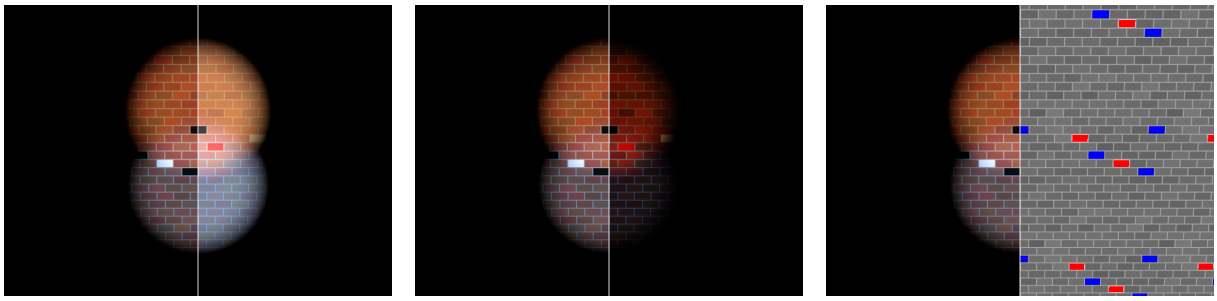


Figure 3: The other three mistakes on the right of the split. **Mode 2, no decode:** washed out, milky. **Mode 3, no encode:** dark and crushed. **Mode 1, `albedo_check`:** the bare albedo with no light. Charcoal bricks are blue and white paint is red. In the test tiles (key `T`, mode 1), the grey ramp is blue up to byte 56 and red from byte 244 on. Check both edges against Equation 1.

Write-up 1.

1. Your no-decode tile shows texel 128 as 188. How many times too much light does a mid-grey wall reflect if the decode is skipped? Work it out from Equation 1. The chapter says "about 2×" and uses $0.5^{2.2}$. Give the exact factor and the factor with $x^{2.2}$.
2. The wall in mode 0. On some brick, each spot on its own displays the byte 150, on both sides of the split. What does the overlap of the two spots display on the left, and what on the right? Where a pool has faded to a quarter of its centre's light, what does each side display? Work both out from Equation 1, then compare your screenshot. Why can an artist working in a gamma pipeline make one light look right, but never two?
3. A roughness map is loaded as `SRGB` by mistake. Is a roughness of 0.5 read as rougher or smoother, and by how much? Why is no image ever wrong this way for a metallic map that only holds 0 and 1?
4. The chapter's Python check (§9.6.2) tests each channel separately against $[0.04, 0.9]$. Yours tests the luminance of each pixel. Find a texel in the test texture that one check flags and the other passes. Which check is right for a saturated red brick, and why?
5. Generate an albedo texture with any image model (or download one), run Part I on it with `--albedo yours.png`, and look at mode 1. What fraction of it is too dark or too bright, and what in the picture caused it? Is there light and shadow painted into it (§9.6.1, "baked lighting")?

5 Part II: Normal maps and the green channel

Run `make normalmap`. You see a smooth grey sphere, turned half a radian about $+y$, under one directional light that circles it. Its normal map, generated on the CPU, is an 8×8 grid of round bumps tiled 8×4 around the sphere, in the OpenGL convention: green points towards the top of the image. The host also generates its DirectX twin, the same bytes with green inverted, and the mesh carries a tangent per vertex, `inTangent.xyz` plus a sign in `inTangent.w`. The map is bound as `tex1` through a `UNORM` view. `tex0` is a plain grey albedo, through an `SRGB` view.

5.1 TASK 2: `normalmap.vert` and `normalmap.frag`

Listing 2: `shaders/normalmap.vert`, after the given `gl_Position`

```
// TODO(TASK 2a)
// The tangent frame in world space. A normal is turned by the normal matrix,
```

```
  // transpose(inverse(mat3(u.model))); a tangent lies along the surface, so it is
  // turned by mat3(u.model) like any direction. B = cross(N, T) * inTangent.w.
  // For now every output is the object-space normal: no tangent frame at all.
  outNormal    = inNormal;
  outTangent   = inNormal;
  outBitangent = inNormal;
```

Listing 3: `shaders/normalmap.frag`: `Ng`, `T`, `B` and `L` are normalised above it, the lighting and the modes are given below it

```
  // TODO(TASK 2b)
  // Read the normal map (tex1) at inUV and decode it to a vector n in [-1, 1]. A DirectX
  // map (u.knob == 1) stores green the other way up: flip n.y. Then take n from
  // tangent space to world space with the matrix whose columns are T, B and Ng,
  // and normalize the result.
  // For now N is the geometric normal: the map is ignored, the sphere is smooth.
  vec3 N = Ng;
```

2a is three lines. 2b is Equation 2 plus the flip. The panel uses the sphere's world normal (mode 1), so a frame that is right in object space but never turned by `u.model` fails: the sphere is rotated on purpose. Keys `C`, `K` and `G` choose which map is bound, how you read it, and through which view. Use them for the write-up, not for passing the panel.

Check values.

- **Six probes**, one on each side of a bump near the centre. The CPU does not use your shader's method at all. It casts a ray through the pixel, moves the sphere's surface out by the same height field the map was made from, and takes the normal of that bumpy surface. Each of your normals must be within $\cos^{-1} 0.98 = 11.5°$ of it.
- **Convention**: the DirectX map read with `knob = 1` gives the same sphere as the OpenGL map with `knob = 0`, 0.00% different. The DirectX map read as OpenGL does not: 57.6% of the sphere changes.
- **Mean** $|n|$, from the CPU, of the map's own mip chain: 1.0000, 0.9979, 0.9645, 0.8902 at levels 0, 2, 4 and 6. Level 6 is one texel per bump. This one is not a check. It is the data for write-up 2.3.

```
$ make normalmap
  probes          your world normal (mode 1) on six bump flanks against the cpu's:
                  the sphere displaced by the same heights, PASS at dot > 0.98
  probe           (462, 328) right       cpu ( 0.85,  0.06,  0.52)  yours ( 0.86,  0.06,  0.51)  dot 1.0000   PASS
  probe           (456, 252) up-right    cpu ( 0.42,  0.82,  0.39)  yours ( 0.41,  0.82,  0.40)  dot 0.9999   PASS
  probe           (382, 253) up-left     cpu (-0.43,  0.79,  0.45)  yours (-0.42,  0.79,  0.45)  dot 1.0000   PASS
  probe           (313, 258) left        cpu (-0.89,  0.21,  0.41)  yours (-0.88,  0.22,  0.42)  dot 0.9999   PASS
  probe           (314, 334) down-left   cpu (-0.50, -0.65,  0.58)  yours (-0.50, -0.65,  0.58)  dot 1.0000   PASS
  probe           (389, 336) down-right  cpu ( 0.34, -0.64,  0.69)  yours ( 0.34, -0.64,  0.69)  dot 1.0000   PASS
  convention      DX map + flip (knob 1) vs GL map    0.00% of the sphere differs   PASS (< 0.5%)
  no flip         DX map, knob 0 vs GL map           57.58% of the sphere differs   PASS (> 10%)
  mean |n|        mip 0  1.0000   mip 2  0.9979   mip 4  0.9645   mip 6  0.8902
                  n = byte / 255 * 2 - 1 over every texel of the level (cpu)
  image           MATCH (0.00% differ)
  gpu time        0.039 ms median, 0.173 ms p95
```

Part II calls `linear_to_srgb` for the screen, so it only matches the reference once TASK 1b is done. The panel says so if it is not.

5.2 Expected result

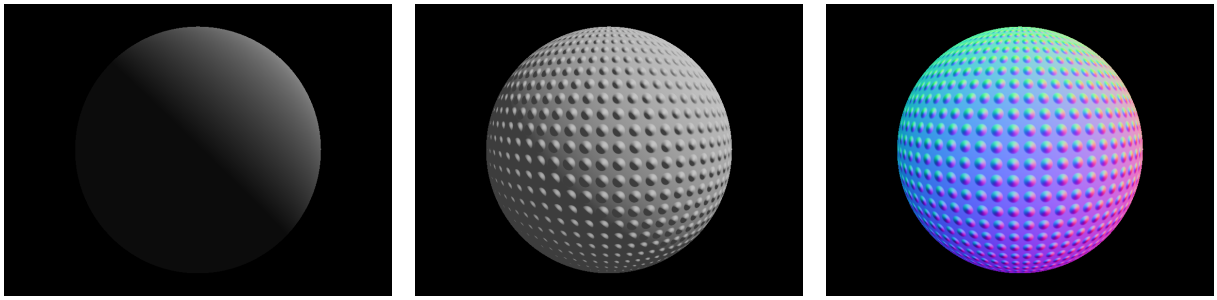


Figure 4: **Skeleton:** the map is ignored and the sphere is smooth (and dark, if TASK 1b is still missing). **Mode 0, done:** OpenGL map, read as OpenGL. The light is up and to the right. **Mode 1:** the world normal as a colour. This is the view the probes read.

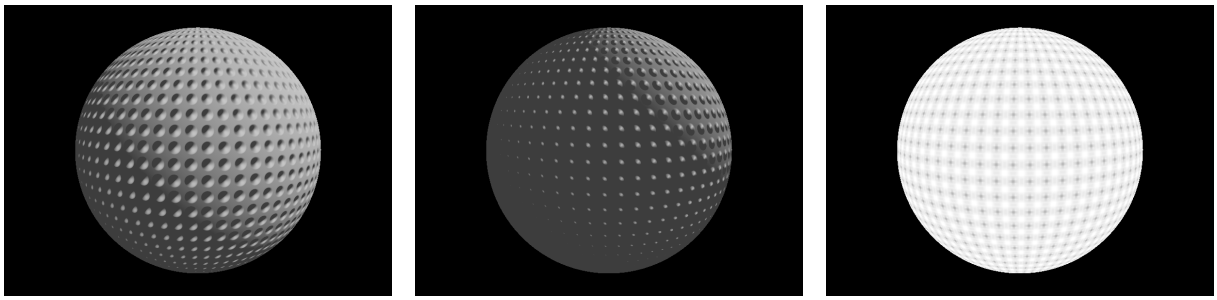


Figure 5: Three ways to get it wrong, for the write-up. **DirectX map, no flip:** the same light, and every bump now reads as a dimple. **Normal map bound as SRGB (key G):** every normal leans the same way and most of the sphere goes dark. **Mode 3 at level 4:** the length of the decoded normal before normalising. White is 1. The grey grid is where neighbouring slopes were averaged into a shorter vector.

Write-up 2.

1. Bind the DirectX map without the flip and let the light circle the sphere (the chapter's 360° test). Describe exactly what goes wrong and *when*: at which light positions do the bumps look right, and at which do they look like dents? Explain this from Equation 2.
2. Bind the correct map as `SRGB` (key `G`). Which way do the normals tilt, and why does a flat area, $(128, 128, 255)$, stop being flat? Compute the normal that this texel turns into.
3. The panel prints the mean length of the decoded normal at mip 0, 2, 4 and 6. Why does it shrink, and why does the chapter say to renormalise normal-map mips? After you renormalise, what has still been lost about the surface?
4. Run the 360° test on a normal map from an image model or from a texture site, with `--normal yours.png`. Which convention is it, and how did the test tell you?

6 Part III: Which mip did the GPU pick?

Run `make floor`. A checkered floor, 400 m across, runs to the horizon, and the camera is 1 m above it, looking 8° down. The texture is 1024×1024, repeats every 2 m, and has all 11 mip levels, box filtered on the CPU. Key `F` cycles the sampler through *nearest* and *bilinear* (both without mips: they always read level 0), *trilinear*, and *anisotropic* $\times 4$ and $\times 16$. You write the

two numbers the GPU computes before it filters: Equations 3 and 4. The views that colour them are given in `floor_helpers.glsl`.

6.1 TASK 3: `miplevel.frag`

Listing 4: `shaders/miplevel.frag`, the two TODOs

```
// TODO(TASK 3a)
// The level of detail: log2 of rho, where rho is the LONGER of the two footprint
// sides (the length of dFdx and of dFdy of the texel position uv * u.texSize).
// Level 0 is one texel a pixel, level 1 two, level 2 four, ...
float lod_of(vec2 uv) {
  return 0.0;
}

// TODO(TASK 3b)
// The anisotropy ratio: the longer footprint side over the shorter one, clamped to
// [1, 16] (16 is the most taps an anisotropic sampler takes). 1 means a square
// footprint; 8 means eight texels along the long side for every one across.
float aniso_ratio(vec2 uv) {
  return 1.0;
}
```

The texel position is `uv * u.texSize`. Take its derivatives, not those of `uv`, or every level is off by $\log_2 1024 = 10$. Mode 1 colours `lod_of` as bands, one colour per level, with a dark line at each whole level. Mode 2 colours `aniso_ratio` as a heat map, blue below 4 and warm above. Mode 3 writes both numbers into a float target for the panel.

Check values.

- **Six probe rows** down the column $x = 600$, from 1.5 to 12.9 m away. The CPU computes the exact footprint: it intersects the rays through the pixel and its neighbours with the floor. Your level must be within 0.05 of it and your ratio within 3%. The `hw` column is the GPU's own answer, `textureQueryLod` with a trilinear sampler. It agrees with the CPU to 0.01, so Equation 3 really is what this GPU does.
- **Ratio above 4** on 37.8% of the floor's pixels, the same as the CPU.
- **The sweep** is a measurement, not a check. Each filter's frame is timed with 16 texture reads per pixel, so that sampling is what costs time. With one read the whole frame takes 0.04 ms whatever the filter, which is noise. Before you look at it, write your prediction in your report: the five filters ranked from cheapest to most expensive, and the cost of $\times 16$ as a multiple of trilinear's, with one line of reasoning.

```
$ make floor
  camera          1 m above a 400 m floor, 8 deg down, 60 deg fov, horizon at row 227
  probes          column x=600, near to far. cpu: exact, from where the rays through
                  the pixel and its neighbours meet the floor. yours: mode 3 of miplevel.frag.
                  hw: textureQueryLod with a trilinear sampler (the raw lod).
  row y=580       cpu lod  1.21 ratio  1.58   yours  1.21  1.58   hw  1.21   PASS    1.5 m away
  row y=500       cpu lod  1.95 ratio  2.05   yours  1.95  2.05   hw  1.95   PASS    1.9 m away
  row y=420       cpu lod  2.95 ratio  2.89   yours  2.95  2.89   hw  2.95   PASS    2.8 m away
  row y=360       cpu lod  4.02 ratio  4.18   yours  4.02  4.18   hw  4.02   PASS    4.1 m away
  row y=310       cpu lod  5.36 ratio  6.65   yours  5.36  6.65   hw  5.36   PASS    6.7 m away
  row y=270       cpu lod  7.23 ratio 12.62   yours  7.23 12.62   hw  7.22   PASS   12.9 m away
  tolerance       lod within 0.05 levels, ratio within 3%;  hw and cpu at most 0.01 levels apart
  ratio > 4       yours on 37.8% of the floor pixels, cpu 37.8%   PASS
  sweep           GPU ms a frame, 16 texture() calls a pixel, 3 rounds
                  filter          median   each round              vs trilinear
                  nearest         ......   (your numbers)
                  ...
  image           MATCH (0.00% differ)   (mode 0, aniso16, 1 texture() call)
```

The image check uses mode 0, which never calls your two functions. For Part III the probes are the test. The image only confirms that TASK 1's encode is in place.

6.2 Expected result

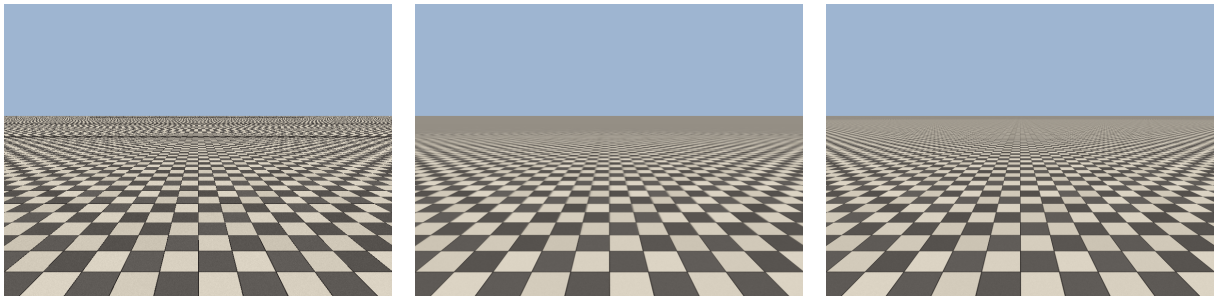


Figure 6: Mode 0 with three filters. **Nearest**, no mips: sharp near the camera, moiré and shimmer towards the horizon, where hundreds of texels share one pixel and the sampler picks one of them. **Trilinear:** the moiré is gone, and so is the far floor, which turns into flat grey. **Anisotropic** ×16**:** the checker stays sharp almost to the horizon. This is the reference image. Bilinear (like nearest) and ×4 (in between) are in `figures/`.

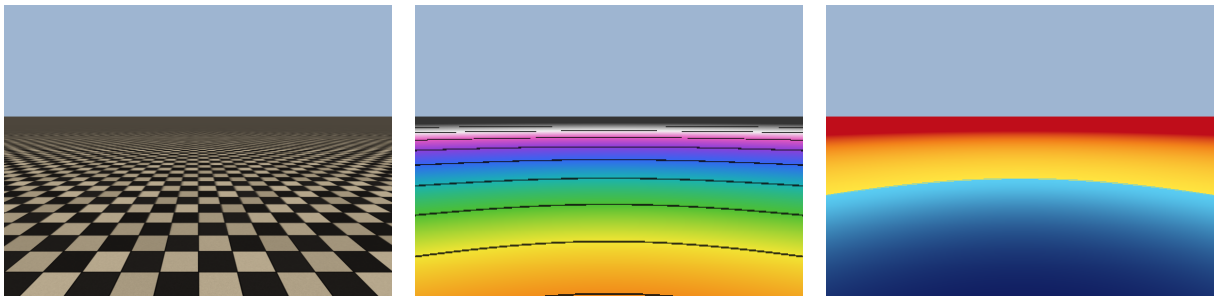


Figure 7: **Skeleton**, mode 0 (dark until TASK 1's encode is in). **Mode 1:** `lod_of` as bands, with a dark line at each whole level. **Mode 2:** `aniso_ratio`. The sharp edge between blue and warm is where the ratio is 4. Compare where it sits with where trilinear went grey.

Write-up 3.

1. In your mode 2 picture, where is the anisotropy ratio above 4? Show that this is the same region where trilinear blurs the grout lines in mode 0. Use Equation 4 to explain why it sits there.
2. Your timing table against the chapter: §9.2.2 says anisotropic ×16 costs about 15% more than trilinear. What does it cost on your GPU, and how many texels does each filter actually read per pixel in the region where the ratio is 16?
3. The mip-level bands in mode 1 are not straight lines across the screen. Why do they bend towards the sides? Answer with Equation 3.
4. Your prediction for the sweep against the measurement. Where were you wrong?

7 Part IV: Transparency without sorting

Run `make oit`. An opaque checkered wall and floor, then everything transparent: three glass panes, 4 m by 3 m with $\alpha = 0.45$, that cut through each other, and 1000 smoke particles between 4 and 30 m from the camera. Key `M` cycles three methods, all drawn in linear light into a float target and encoded for the screen once at the end:

- **unsorted**: Equation 5 in whatever order the objects were generated, with the depth test on and depth writes off.
- **sorted**: the CPU sorts the panes and particles back to front by their centres every frame, then draws them the same way.
- **wboit**: your two shaders. Pass 1 draws every transparent fragment once, into an `RGBA16F` accumulation target with additive blending and an `R16F` revealage target with blend factors $(0, 1 - \text{src})$, so that the target ends up holding $\prod(1 - \alpha_i)$. Pass 2 is a fullscreen triangle that turns the two into a colour and an alpha, and blends that over the opaque scene.

The blend states, the two targets and the depth they test against are C++. Both of your shaders get `surface()` from the given `transparent.glsl`, which is the colour and straight alpha of the current fragment. The three methods shade identically, so any difference between them comes from compositing alone.

7.1 TASK 4: `wboit_accum.frag` and `wboit_composite.frag`

Listing 5: `shaders/wboit_accum.frag`

```
void main() {
  vec4 c = surface();

  // TODO(TASK 4a)
  // c is this fragment's straight colour c.rgb (linear) and alpha c.a.
  // The weight: w = clamp(0.03 / (1e-5 + (z / 200)^4), 1e-2, 3e3), with z the
  // VIEW-SPACE depth in metres (inViewDepth), not the window depth gl_FragCoord
      .z.
  // accum: the colour premultiplied by alpha, times w, and alpha times w.
  // reveal: the alpha, so the blend leaves the product of (1 - alpha).
  // Until then nothing is added and nothing is covered: WBOIT shows the opaque
      scene.
  outAccum  = vec4(0.0);
  outReveal = vec4(0.0);
}
```

Listing 6: `shaders/wboit_composite.frag`

```
void main() {
  ivec2 p = ivec2(gl_FragCoord.xy);

  // TODO(TASK 4b)
  // Read accum (tex0) and reveal (tex1, .r) at p with texelFetch.
  // Where nothing transparent was drawn (reveal 1), leave the scene alone.
      Elsewhere
  // output the weighted average colour accum.rgb / accum.a (keep the division
      away
  // from 0) with alpha 1 - reveal.
  // Until then alpha 0: the scene stays as it is.
  outColor = vec4(0.0);
}
```

4a is Equation 7 and two writes. 4b is the fraction in Equation 6, and the composite's *over* blend supplies the rest of it. In `accum` the blend adds, so order cannot matter. In `reveal` it multiplies, so order cannot matter there either. That is the whole trick.

Check values.

- **Single layer**: one pane alone over the scene. WBOIT must equal *over* to within one level, because Equation 6 with one layer *is* Equation 5. Check that on paper first.

- **Two layers**, which tests your weight. Red with $\alpha = 0.3$ at 4 m in front of blue with $\alpha = 0.6$ at 20 m. Red's share of `accum.a` is 0.844, and the revealage is $(1 - 0.3)(1 - 0.6) = 0.280$. With the same weight for every layer, red's share would be 0.333.
- **Order**: the transparent list drawn backwards. WBOIT changes by at most one level, from fp16 sums rounding differently. Unsorted *over* changes on about half of the pixels, by more than 100 levels.
- **vs sorted**, **weight**, **fp16** and **timing** are measurements for your write-up. Before you run the timing, write your prediction in your report: from 1000 to 10000 particles, which method's time grows the most, and why. The weight line is the CPU evaluating Equation 7 over the depth range of the transparent objects, once with metres and once with the depth buffer's value, the way the chapter's code does.

```
$ make oit
  scene           opaque wall and floor; 3 glass panes 4 x 3 m, alpha 0.45, cutting
                  through each other; 1000 smoke particles 3 .. 30 m from the camera
  method          wboit, 1000 particles
  single layer    one pane over the scene: max |wboit - over| 0 levels   PASS (<= 1)
  two layers      red alpha 0.30 at 4 m in front of blue alpha 0.60 at 20 m, pixel (400, 120)
                  red share of accum   yours 0.844   cpu 0.844   (one weight for all 0.333)
                  revealage            yours 0.280   cpu 0.280   PASS
  order           transparent list drawn in reverse: wboit differs on 6.00% of pixels,
                  by at most 1 level (0.00% by more than 1)   PASS
                  unsorted Over reversed the same way: 50.81% of pixels, by up to 142
  vs sorted       mean |wboit - sorted| 3.9 levels, max 84 (report only)
                  the same with the chapter's weight: mean 3.9, max 87
                  mean |unsorted - sorted| 5.9 levels, max 114
  weight          w over the transparent objects, 4.3 .. 30.0 m from the camera
                  chapter, window depth 0.0023 .. 0.0223   w 3000.0 .. 3000.0
                  view depth in metres                     w 58.2 .. 2937.3
                  (a depth buffer with 0 near, 1 far: z 0.9777 .. 0.9977, w 2999.8 .. 2999.8)
  fp16 accum      largest accum.a, and how many pixels have a channel at 65504
                  ...
  timing          GPU ms of the transparent passes (median of 3 rounds); sort on the CPU
                  ...   (your numbers)
  image           MATCH (0.00% differ)
```

The depth buffer in this lab is reversed-Z: 1 at the near plane, 0 far away. The panel also shows the other convention, 0 near and 1 far, so that you cannot blame the result on it.

7.2 Expected result

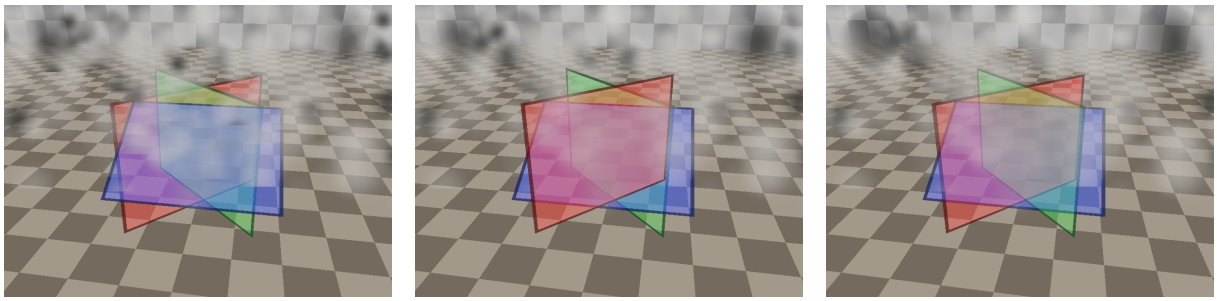


Figure 8: **Unsorted:** blue was drawn last, so it covers red and green everywhere, and dark smoke from behind the panes is painted over the glass. **Sorted by object:** red's centre is nearest, so red is drawn last and lies on top everywhere, which is wrong wherever red passes behind the other two. **WBOIT:** no pane is on top. The overlaps are weighted averages, yellow where red and green cross and a neutral lavender where all three overlap.

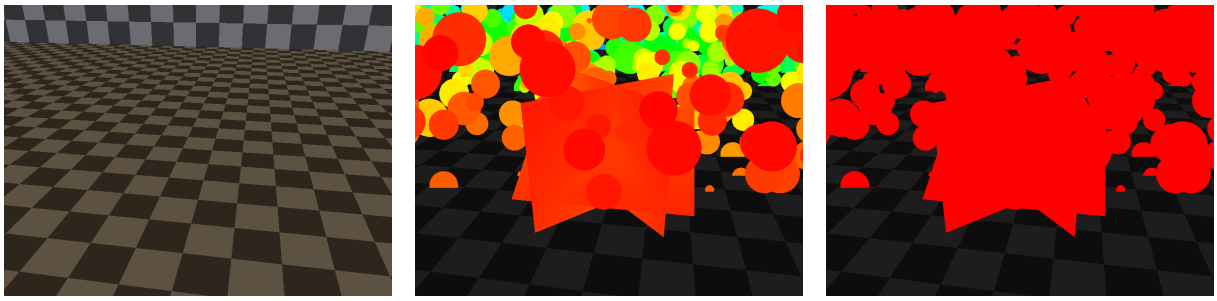


Figure 9: **Skeleton:** WBOIT adds nothing, so only the wall and the floor are left. **Mode 3,** the weight of the front-most layer, $\log_{10} w$ from blue (30) to red (3000): the panes and the near smoke are red, and the far smoke goes through yellow and green to cyan. **Mode 3 with key W,** the weight computed from `gl_FragCoord.z` as in the chapter: red everywhere.

Write-up 4.

1. The chapter's WBOIT shader (§9.5.3) computes the weight from `in.position.z`, the depth in the depth buffer, which is between 0 and 1. Put that into Equation 7. What weight does every fragment get? Show it with the panel's `weight` line and a mode 3 screenshot of both weights (key `W`), then say what the chapter's version of WBOIT reduces to. In this scene, the two pictures differ by only a few levels on average. Why so little here? Then run `make oit ARGS="--particles 40000 --mode 1"` with each weight (key `W`). Magenta marks pixels where fp16 accumulation saturated at 65504. Which weight saturates, and why?
2. Two layers of $\alpha = 0.5$, red in front of blue (Section 3). Prove that weights in the ratio $w_{\text{front}} : w_{\text{back}} = 2 : 1$ make WBOIT exact. For n layers, what should the ideal weight of a layer be? Why can no function of z alone produce it?
3. The three glass panes cut through each other. Why can no back-to-front order of the three *objects* be right? Point to the place in your sorted screenshot where it fails.
4. Your timing table: unsorted, sorted (the sort on the CPU plus the GPU) and WBOIT, at 1000 and 10000 particles. Which grows fastest, and why? Was your prediction right?
5. Find a place in your WBOIT picture where it is visibly wrong compared to the sorted one. Say why, using Equation 6.

8 Submission

Submit your `shaders/` directory and a report, in any format you like (Google Docs, Word, PDF), containing:

- **Six shader files**: `color.glsl`, `normalmap.vert`, `normalmap.frag`, `miplevel.frag`, `wboit_accum.frag` and `wboit_composite.frag`. Nothing else in the repository should have changed.
- **Your machine**: the `make info` screenshot, with every TASK line reading `done`. Every number you report is about your machine, not about GPUs in general.
- **Predictions**: for the Part III filter sweep and the Part IV timing, written in the report *before* you ran the measurement, and left as you wrote them. A wrong prediction costs nothing. A missing one, or one written afterwards, does.
- **Screenshots**: the panel of each of the four parts, plus the screenshots that the write-ups ask for.

- **Four write-ups**: the numbered questions under each part. Answer from what is on your screen and the equations here, not from a general description of the topic. Several of them disagree with the chapter on purpose. A number that contradicts the chapter, reported honestly and explained, is a good answer. A number that agrees with the chapter with no evidence is not.
- **AI prompt log**: one entry per prompt, each with the **prompt**, the **model**, the **idea behind it** (what you needed and what you already knew) and the **verification** (how you checked it: the check values here, the panel's CPU column, the reference image, working a value out on paper).

A log entry like "wrote the WBOIT shader for me" with empty idea and verification fields does not meet this requirement. You may ask AI to write a shader. The write-ups and those two fields are where you show your own thinking. You should be able to answer a follow-up question in person about anything you submit, and in this lab you will: the last ten minutes of the session are for that.

9 References

- Akenine-Möller, T., Haines, E., Hoffman, N. (2018): *Real-Time Rendering*, 4th ed., Ch. 5.5 (transparency and compositing), Ch. 6 (texturing, mipmaps, anisotropic filtering) and Ch. 8.1 (sRGB and linear values).
- Williams, L. (1983): *Pyramidal Parametrics*, SIGGRAPH '83. The mip chain of Part III.
- Porter, T., Duff, T. (1984): *Compositing Digital Images*, SIGGRAPH '84. The *over* operator.
- McGuire, M., Bavoil, L. (2013): Weighted Blended Order-Independent Transparency, JCGT 2(2). Equation 7 is their Equation 9. Read what they mean by z.
- Mikkelsen, M. (2008): *Simulation of Wrinkled Surfaces Revisited*. The tangent space that most tools bake normal maps in, and why the bitangent's sign is stored separately.
- Vulkan Specification, §Texture Level-of-Detail Operation (Equation 3, and the freedom drivers have in computing it), and §sRGB EOTF.